

Threads e Concorrência

Sistema Bancário Concorrente em Python

Pitch Técnico - Guia Completo

Equipe e Repositório

Autores do Projeto

- Franklin Gaio Figueira Filho (1230115176)
- Maria Luiza França Mendes (1230112799)
- Jorge Luiz Furtado Maia (1230113709)
- Daniela Moreira Negreiros Dias (1230203973)

Acesso ao Código



github.com/franklin808/Thiago-UVA/tree/main

Código fonte comentado, guias de explicação técnica e simulação
concorrente ao vivo.

O Problema Real

Em sistemas bancários reais, múltiplos usuários (aplicativo, caixa eletrônico, agência) acessam a mesma conta simultaneamente.

Sem um mecanismo rigoroso de controle, essas operações simultâneas disputam o mesmo recurso financeiro, resultando em dados inconsistentes e graves prejuízos operacionais.



Entendendo a Race Condition

- Ocorre quando duas ou mais threads manipulam uma variável compartilhada sem sincronização.
- **Exemplo Crítico:** Uma conta com R\$ 1.000. Duas threads sacam R\$ 100 simultaneamente.
- Ambas leem o saldo original (R\$ 1.000) antes que a outra conclua a atualização.
- O resultado é subscrito, ignorando efetivamente um dos saques.

$$S = 1000 - 100 = 900(\text{Incorreto})$$



Terminais com concorrência de dados

Solução: Lock e Exclusão Mútua

- A classe **ContaBancaria** atua como a memória compartilhada do sistema.
- Utilizamos o `threading.Lock()` como mecanismo central de exclusão mútua.
- Quando uma thread entra no bloco crítico (ex: alterar saldo), ela "adquire" a trava.
- Outras threads que tentarem acessar a mesma área ficam bloqueadas aguardando a liberação.



O Que o Lock Garante?

-  **Atomicidade:** A leitura do valor atual, a realização do cálculo matemático e a escrita do novo saldo ocorrem como um bloco único e ininterrupto.
-  **Visibilidade:** Assim que a trava (Lock) é liberada, a modificação do saldo fica imediatamente visível para todas as outras threads em execução.
-  **Segurança de Exceções:** Utilizando o contexto `with self.lock:`, a trava é liberada automaticamente ao fim do processo, mesmo que ocorra uma falha crítica.

Comunicação com Queue

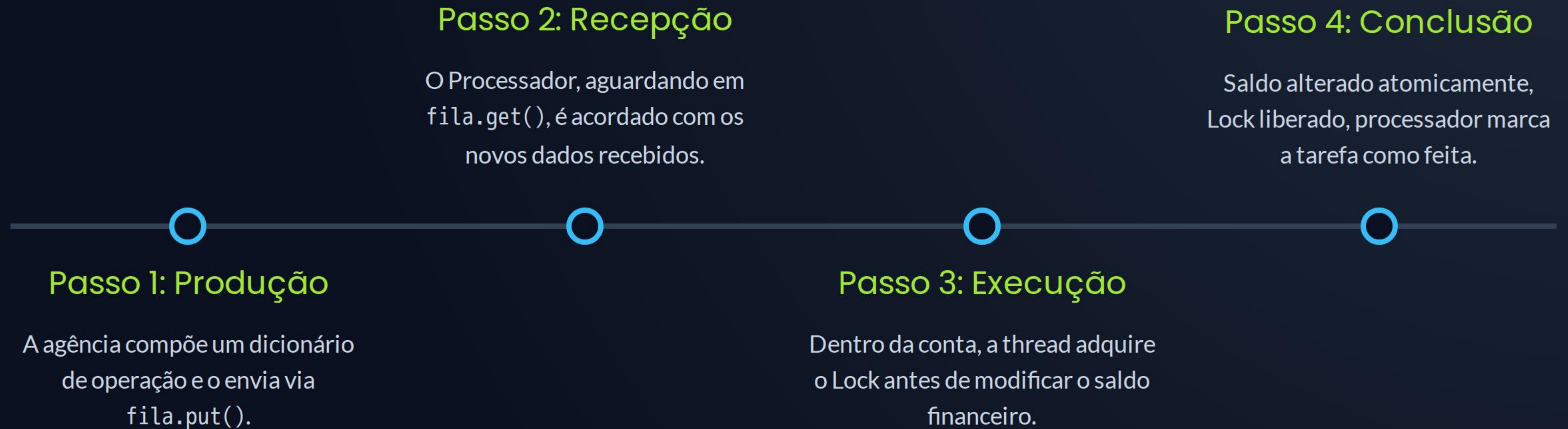


Padrão Produtor-Consumidor

A `queue.Queue()` é uma estrutura naturalmente thread-safe. As Agências (produtoras) geram dados e os colocam na fila de transações.

O Processador Central (consumidor) retira esses dados um a um e os aplica. O processador entra em estado de espera, sem consumir CPU de forma ociosa, até que uma nova transação esteja disponível.

Fluxo de Uma Transação



Execução ao Vivo

Nosso projeto utiliza apenas bibliotecas nativas do Python (threading, queue, time, random), dispensando instalações complexas.

Na demonstração de terminal, ilustramos primeiramente a falha financeira da Race Condition desprotegida, seguida do processamento simultâneo e seguro de diversas transações concorrentes.



Glossário de Concorrência



Thread

Linha de execução paralela dentro do programa. Diferente de um processo, threads compartilham a mesma memória do sistema, permitindo operação simultânea e rápida alteração de estado.



Seção Crítica

Trecho sensível de código onde variáveis compartilhadas são lidas ou escritas. Exige proteção rigorosa para evitar inconsistências durante acesso paralelo inoportuno.



Queue Segura

Fila thread-safe que abstrai a sincronização manual. Funciona como um intermediário eficiente e bloqueante para a comunicação fluida entre processos produtores e consumidores.

Armadilhas Avançadas

O GIL do Python

Apesar do Global Interpreter Lock (GIL) assegurar que apenas uma thread execute o bytecode por vez, ele **não** garante a atomicidade nas operações compostas ("ler-calcular-escrever"). Em processamentos de I/O, o GIL é liberado, exigindo a inclusão estrita do Lock explícito.

Riscos de Deadlock

O temido Deadlock ocorre quando duas ou mais threads ficam bloqueadas eternamente, esperando pela trava umas das outras. Nosso modelo é imune a esse cenário, pois cada método gerencia e libera exclusivamente uma única trava imediata.

Resumo Técnico Geral

Conceito Chave	Descrição Técnica	Aplicação no Nosso Sistema
Memória Compartilhada	Variável acessada simultaneamente.	Objeto ContaBancaria (self.saldo)
Exclusão Mútua (Lock)	Trava de segurança que isola o contexto.	Proteção nas funções depositar() e sacar()
Produtor-Consumidor	Separação de tarefas através de fila.	Agências inserem dados, Processador aplica.
thread.join()	Sincronização de encerramento.	Main aguarda finalização para evitar aborto.

Perguntas?

Obrigado pela atenção!

 Repositório: github.com/franklin808/Thiago-UVA/tree/main

